Assignment 3 – Practical Deep Learning Workshop

david podvinsky-215452079

Github: https://github.com/davpod/deep_learn_HW3

Overview: We are given a home depot search-product relevancy list, and our task is to build a model that would be able to give a relevancy score between 1-3 for each pair of search term and product description, in the instructions that were given to the human scorers, we can learn that relevancy is based upon search word relevancy and also brand name, which is little to use for me as of now since I don't know how to extract brand name nor am I willing to do it, and thus my models will have to figure it out themselves.

in the Kaggle competition page, we can clearly see that the winner got 0.413 test score, and according to the competition page is RMSE, but upon closer inspection, we can see that for other participants, manual test RMSE score calculations also tend to be very high, meaning that the Kaggle test score may also actually be MAE or some other metric, which would mean that we cant really use it as a baseline.

Anyway we can see that the top of the minds, with much more resources, motivation and techniques still get at most 0.413 MAE which is not really that great, leading me to believe that this problem is semantically capped, meaning that there will only be so much we can do, and that even the best model would probably not be able to solve this problem.

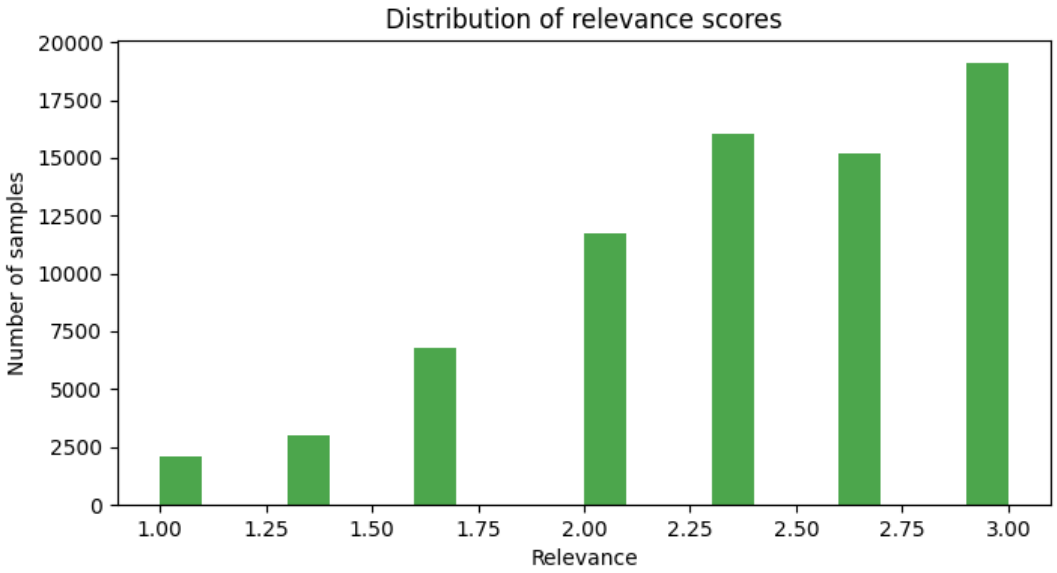
Eda: our first step in any data related task, will of course be Data science, evaluating and learning the data we are working on, identifying potential problems or clues it may hold, and how should we preprocess and clean it.

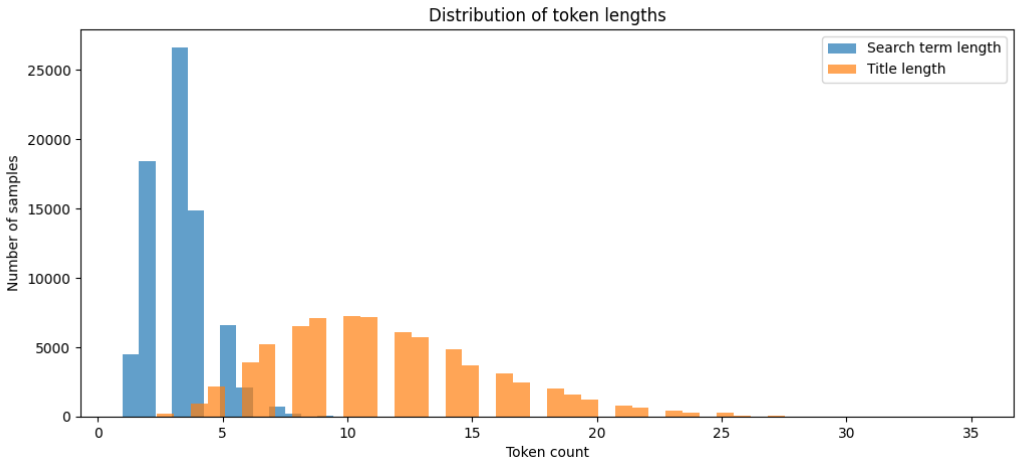
By running the code in the EDA.py file, we get the next data:

Training set size: 74067, Max search term length: 14, Max product title length: 35, Unique search tokens: 8433

Unique title tokens: 28121

And the 2 plots for relevancy distribution, and terms length distribution:





As we can see, our data is quite unbalanced with most of the data being on the relevant side, meaning that irrelevant searches are quite underrepresented.

Also, we can see that while product descriptions are long averaging at around 11 words per title, in contrast our search term are very short, averaging at around 3 words per search.

This asymmetry may cause us difficulties as we essentially have 2 different input lengths, meaning our model must handle asymmetric input lengths.

Another problem we can see is that titles have a lot of unique words, meaning there will be a lot of rare words that the model will barely see and struggle to remember in word embeddings. We have 18208 unique words with less than 5 appearances in the titles.

We can also see that we have 74 unique characters in the train set, meaning that a character based encoding model may have a better chance of learning exotic symbols and relationships. This is even easier to see in per category character count having 51 unique characters in search terms(which is still a lot considering normal humans wrote those things), and 71 unique characters in product titles, meaning that there is a huge character overlap between the 2 fields probably making it easier for a character based model to deal with.

We are also provided 2 test files: test.csv and solution.csv, where test includes title+search, and solution includes the actual relevancy score, and to make our like easier, we would want it to be in 1 singular file, to achieve it we run: merge_test_split.py

Also solutions.csv contains a lot of missing data, so for each relevancy: -1 we would have to drop it away.

Character based encoding:

Preprocessing: for this section, we want a character based encoding, to do it we will have a function that will have a simple function that given an array of datasets, it will simply run go column by column, identifying all the characters, and if not encountered already just add it to a dictionary. This way we find out that we have 87 unique characters in total.

TASK1_SIAME.py

TASK1_BASELINE.py

Model: a siamese model requires an independent model to work as its encoder, so essentially we want: siamese model that uses the input from 2 instances of its encoder, and an encoder that can process this array of encoded characters. In our case our inputs would be the search term and the product title, and will output relevancy.

Encoder: for simplicity and because I love speed, I will just use a 1d cnn:

```
class CharEncoder(nn.Module):  1 usage  david podvinsky *
    def __init__(self, vocab_size, embed_dim=32, dropout=0.2):  david podvinsky *
        super().__init__()
        self.emb = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.c1 = nn.Conv1d(embed_dim,  out_channels: 64, kernel_size=3, padding=1)
        self.c2 = nn.Conv1d( in_channels: 64,  out_channels: 128, kernel_size=3, padding=1)
        self.fc = nn.Linear( in_features: 128,  out_features: 128)
        self.dropout = nn.Dropout(dropout)  # controlled via hyperparameter

    def forward(self, x):  david podvinsky *
        x = self.emb(x).permute(0, 2, 1)  # [batch, embed_dim, seq_len]
        x = F.relu(self.c1(x))
        x = F.relu(self.c2(x))
        x = F.adaptive_max_pool1d(x,  output_size: 1).squeeze(-1)
        x = self.dropout(self.fc(x))  # apply dropout after FC
        return x
```

My experiments shown that trying to make the model any more complex, will yield to a very quick overfitting, or simply wont yield any better results, so I have settled down with this tiny model.

Siamese: a simple model that uses char encoder twice, concats the outputs and runs it thru a final fully connected layer.

```
class SiameseRegression(nn.Module):  1 usage  david podvinsky *
    def __init__(self, vocab_size):  david podvinsky *
        super().__init__()
        self.encoder = CharEncoder(vocab_size,dropout=DROPOUT)
        self.fc = nn.Linear( in_features: 256,  out_features: 1)

    def forward(self, s, t):  david podvinsky
        es = self.encoder(s)
        et = self.encoder(t)
        return self.fc(torch.cat( tensors: [es, et],  dim: 1)).squeeze(1)
```

Best scores are with hyper params:

BATCH_SIZE = 128
EPOCHS = 12
LR = 1e-4
DROPOUT=0.2
MAX_SEARCH_LEN = 30
MAX_TITLE_LEN = 50

And the actual scores are:

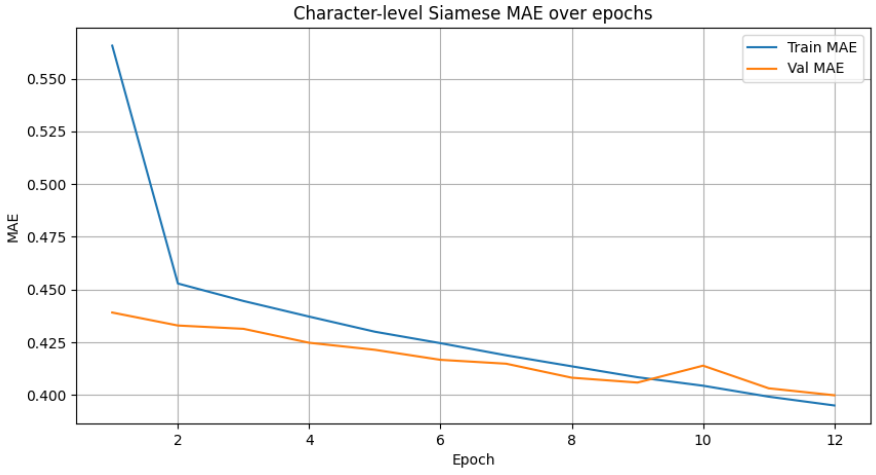
Epoch 12 | Train RMSE: 0.4867 | Train MAE: 0.3950

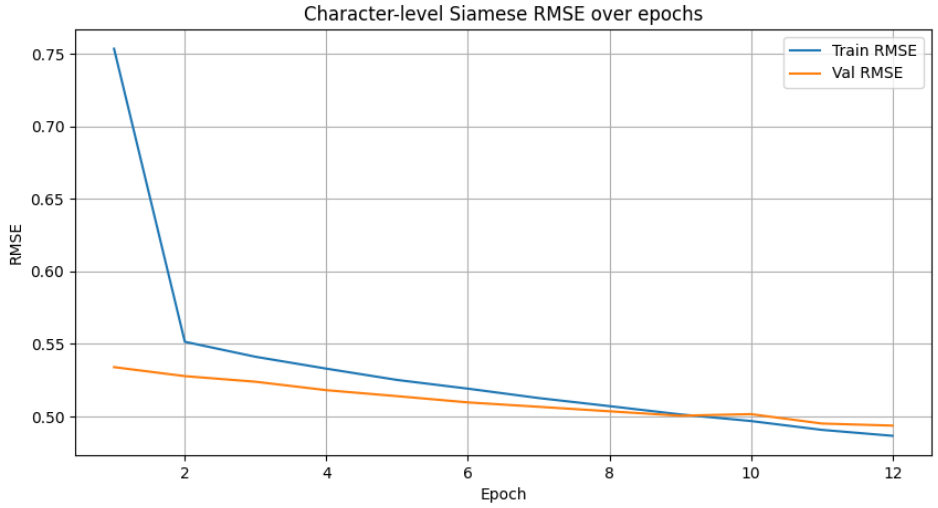
Epoch 12 | Val RMSE: 0.4937 | Val MAE: 0.3999

FINAL TEST RMSE: 0.5235 | Test MAE: 0.4243

Total Neural Model Runtime (Train + Inference): 48.02 sec

With a tight relation between train and evaluation meaning that overfitting is minimal:





<u>Naïve benchmark:</u> TASK1_BASELINE.py

as in previous assignments, we would want a naïve strong benchmark to be able to judge our progress and results.

As a benchmark, we implemented a character-level CountVectorizer (3–5 character n-grams, min frequency = 5) followed by Ridge regression. This gives a simple, interpretable baseline against which to compare the performance of our Siamese CNN models.

Baseline Validation RMSE: 0.6367 | MAE: 0.4960

Baseline Test RMSE: 0.7563 | MAE: 0.5930

Total Baseline Runtime (Train + Inference): 54.68 sec

Which is slower, but expected as it runs on cpu while the models run on the gpu, and the scores are not great, but strong enough.

Word/byte-pair embeddings and word/byte-pair level model

TASK2_MYMODEL.py

Preprocessing: We tokenize search phrases and product titles into words or character-combinations (e.g., "¾", "90°", "#SC"). This ensures both standard words and special product symbols are represented. Each token is mapped to an integer index based on a Word2Vec-trained vocabulary. Sequences are padded or truncated to MAX_SEQ_LEN=50 for uniform input to the model, as learned in the eda, the terms may reach 35 in training alone, and just to be safe we want a bit longer.

Embedding: We trained a Word2Vec embedding on the training corpus (search terms + product titles) with embedding dimension 100. The embedding matrix is initialized with these vectors, and unknown tokens are mapped to <PAD>.

Encoder: this time the same encoder gave quite the weak results, so I took the opportunity to make a better encoder model.

```
class WordEncoder(nn.Module):  1 usage  david podvinsky
    def __init__(self, embedding_matrix, out_channels=128, dropout=0.2):  david podvinsky
        super().__init__()
        vocab_size, embed_dim = embedding_matrix.shape
        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(embedding_matrix, dtype=torch.float32)
        )

        # Multi-kernel CNNs
        self.convs = nn.ModuleList([
            nn.Conv1d(embed_dim, out_channels, kernel_size=k, padding=k // 2)
            for k in [2, 3, 4]  # n-gram sizes
        ])

        self.fc = nn.Linear(len(self.convs) * out_channels,  out_features: 128)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):  david podvinsky
        x = self.embedding(x).permute(0, 2, 1)  # [batch, embed_dim, seq_len]
        conv_outs = []
        for conv in self.convs:
            c = F.relu(conv(x))
            c = F.adaptive_max_pool1d(c,  output_size: 1).squeeze(-1)  # max over sequence
            conv_outs.append(c)
        x = torch.cat(conv_outs, dim=1)  # [batch, out_channels * num_kernels]
        x = self.dropout(x)
        return self.fc(x)
```

Siamese: the same thing except now constrict the output between [1,3]

```python
class SiameseWordNet(nn.Module):  1 usage  david podvinsky *
    def __init__(self, embedding_matrix):  david podvinsky
        super().__init__()
        self.encoder = WordEncoder(embedding_matrix)
        self.fc = nn.Linear( in_features: 256,  out_features: 1)

    def forward(self, s, t):  david podvinsky *
        es = self.encoder(s)
        et = self.encoder(t)
        x = self.fc(torch.cat( tensors: [es, et], dim=1))
        x = torch.sigmoid(x) * 2 + 1  # maps 0->1, 1->3
        return x.squeeze(1)  # <-- Option 2: ensures output shape is [batch]
```

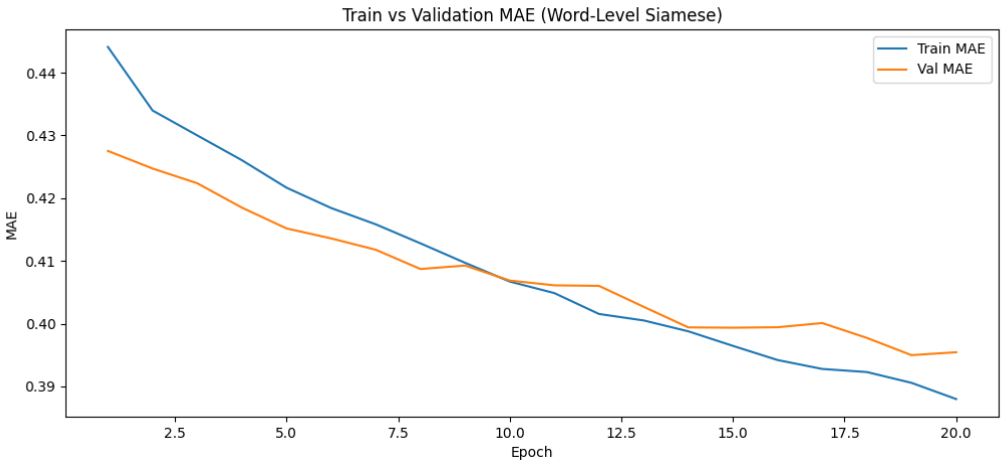
BATCH_SIZE = 128
EPOCHS = 20
LR = 1e-4
DROPOUT = 0.5

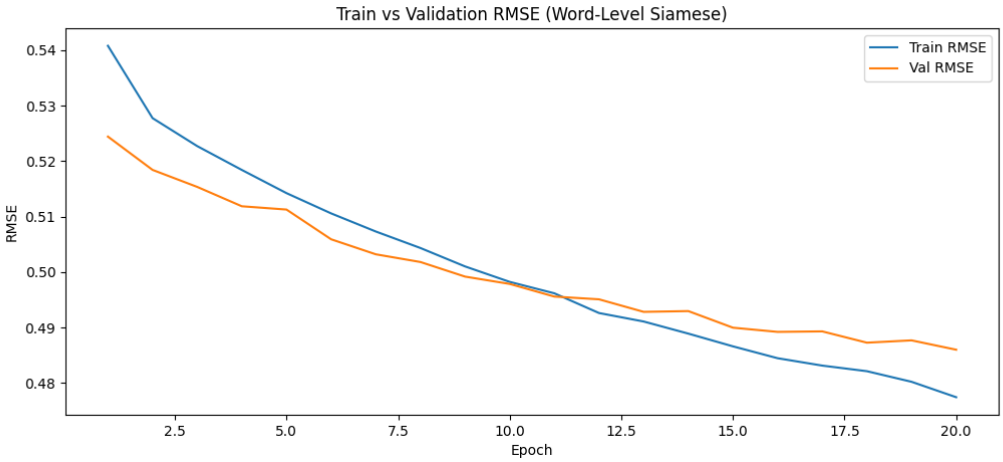
Scores:

Epoch 20 | Train RMSE: 0.4775 | Val RMSE: 0.4860 | Train MAE: 0.3880 | Val MAE: 0.3954

Test RMSE: 0.5179 | Test MAE: 0.4233

Total Runtime (Train + Inference): 83.91 sec





We can see a slight overfitting, but its manageable. Overall just a tiny bit better than the character level model.

feature extractors:

word endcodings: first I will use bert, as that what the LLM suggests

TASK2_BERT.py

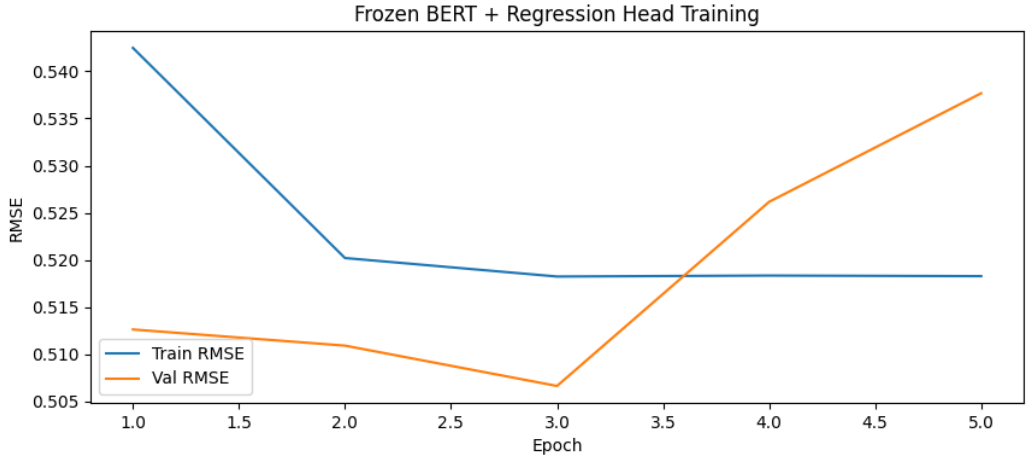
It was definitly a mistake to use bert, even with all its layers frozen, it took more than 10 minutes to train 5 epochs... and in the end we get the result:

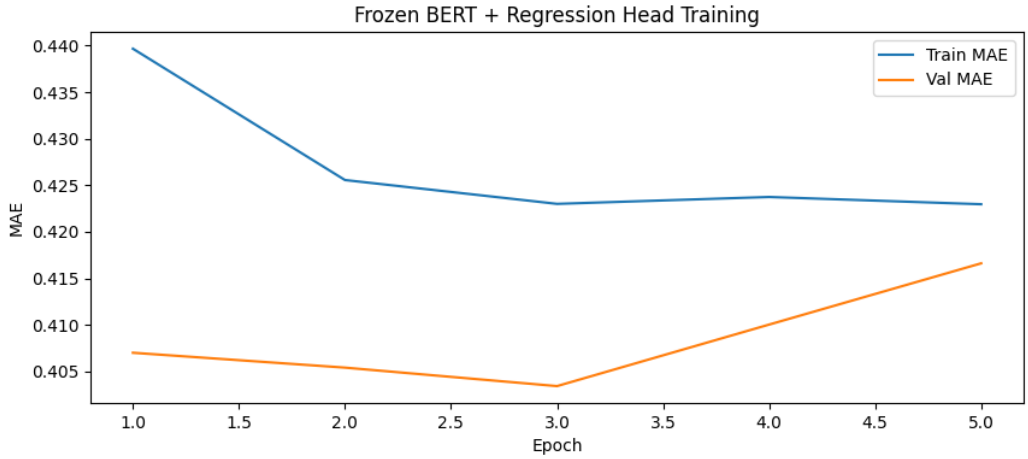
Epoch 5 | Train RMSE: 0.5183 | Val RMSE: 0.5377 | Train MAE: 0.4230 | Val MAE: 0.4166

Test RMSE: 0.5406 | Test MAE: 0.4197

Total Runtime (Train + Inference): 631.48 sec

Im sure there are things to be done, but im not willing to wait so long





Is it overfitting or what is going on there?

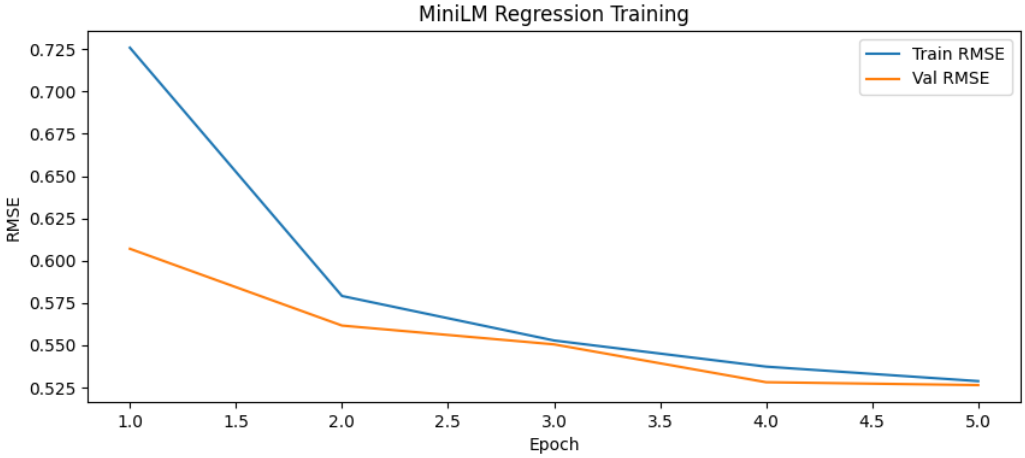
<u>Second model</u>: TASK2_MINILM.py

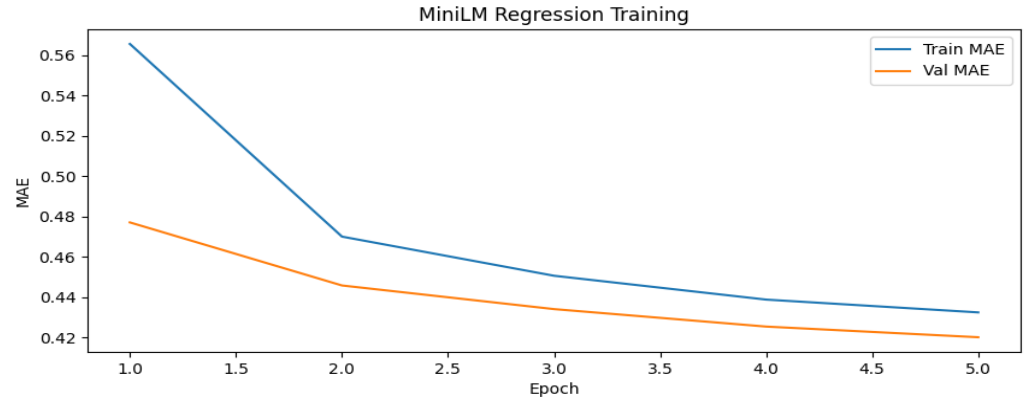
miniLM, 5 times smaller than bert which is awesome

Epoch 5 | Train RMSE: 0.5288 | Val RMSE: 0.5265 | Train MAE: 0.4324 | Val MAE: 0.4201

Test RMSE: 0.5307 | Test MAE: 0.4248

Total Runtime: 153.31 sec





Quite the fitting, so we are free to play more, but I don't really have the time.

Character embedding models:

Model1: my cnn from task1+ ridge head

TASK2_MYCNN_FE.py

Epoch 10 | Train RMSE: 2.1847 | Val RMSE: 2.1784

Ridge Train RMSE: 0.4634 | MAE: 0.3734

Ridge Val RMSE: 0.4973 | MAE: 0.4018

Ridge Test RMSE: 0.5327 | MAE: 0.4312

Ridge Runtime: 0.07 sec

For some reason deleting the fc retraining part sends me to bert training so ill leave it there as its still minimal run time anyway

Model 2: TASK1_SIAME_BILTSM.py

since we want 2 models, and chat doesn't seem to be able to suggest any model, It's a great opportunity to try another encoder architecture for the siame model, I really want to try a transformer, but before that I will go with a safer option of BiLTSM, which from my understanding is 2 LTSM layers whose output is then combined, 1 layer processes the sequential data forward, while the other one processes it In backward direction, meaning we process the data in both a forward and backwards pass at the same time, which In theory should capture dependencies better than a cnn.
FIRST ATTEMPT: just plug in the code chat gave me:

```
class CharBiLSTMEncoder(nn.Module):  1 usage
    def __init__(self, vocab_size, embed_dim=32, hidden_dim=64, dropout=0.2):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(
            embed_dim,
            hidden_dim,
            batch_first=True,
            bidirectional=True
        )
        self.fc = nn.Linear(hidden_dim * 2,  out_features: 128)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        x = self.emb(x)                  # [B, L, E]
        _, (h_n, _) = self.lstm(x)       # h_n: [2, B, H]
        h = torch.cat( tensors: [h_n[0], h_n[1]], dim=1)  # [B, 2H]
        h = self.dropout(self.fc(h))
        return h

class SiameseRegression(nn.Module):  1 usage
    def __init__(self, vocab_size):
        super().__init__()
        self.encoder = CharBiLSTMEncoder(
            vocab_size,
            embed_dim=32,
            hidden_dim=64,
            dropout=DROPOUT
        )
        self.fc = nn.Linear( in_features: 256,  out_features: 1)

    def forward(self, s, t):
        es = self.encoder(s)
        et = self.encoder(t)
        return self.fc(torch.cat( tensors: [es, et],  dim: 1)).squeeze(1)
```

The Siamese head has remained unchanged, with the hyper params:

BATCH_SIZE = 128
EPOCHS = 12
LR = 1e-4
DROPOUT=0.2
MAX_SEARCH_LEN = 30
MAX_TITLE_LEN = 50

We get:

Epoch 12 | Train RMSE: 0.5319 | Train MAE: 0.4357

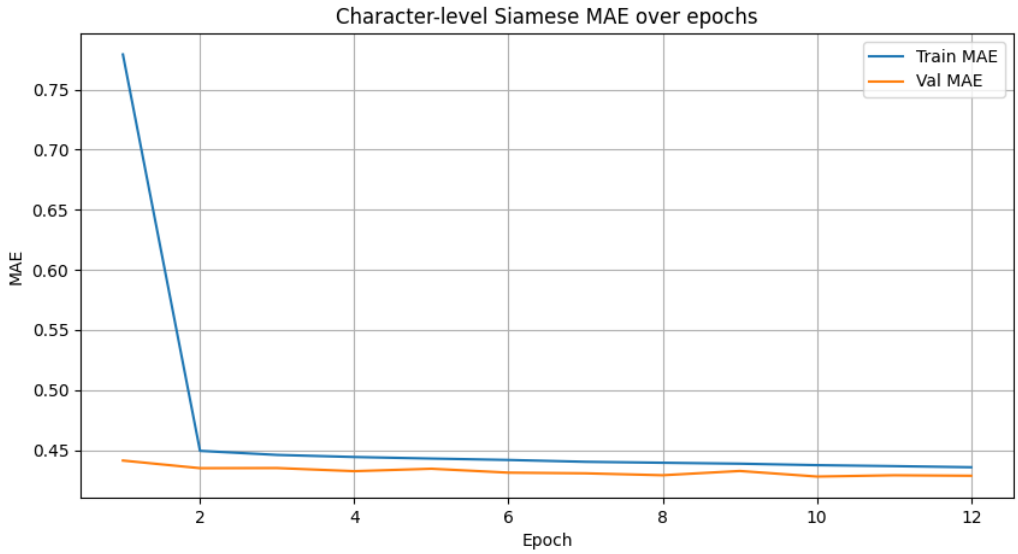
Epoch 12 | Val RMSE: 0.5225 | Val MAE: 0.4287

FINAL TEST RMSE: 0.5294 | Test MAE: 0.4349

Total Neural Model Runtime (Train + Inference): 52.36 sec

Which is on par with the cnn Siamese model, running around the same speeds, except its even more uniform with the deviation between val and test scores being minimal, and also test score is better than last train score?

But the strange things don't even stop here, if we look at the graphs and the training process:



```
Epoch 1 | Train RMSE: 1.0659 | Train MAE: 0.7793
Epoch 1 | Val RMSE: 0.5371 | Val MAE: 0.4413
Epoch 2 | Train RMSE: 0.5479 | Train MAE: 0.4494
Epoch 2 | Val RMSE: 0.5310 | Val MAE: 0.4350
Epoch 3 | Train RMSE: 0.5438 | Train MAE: 0.4460
Epoch 3 | Val RMSE: 0.5298 | Val MAE: 0.4351
Epoch 4 | Train RMSE: 0.5416 | Train MAE: 0.4443
Epoch 4 | Val RMSE: 0.5286 | Val MAE: 0.4325
Epoch 5 | Train RMSE: 0.5402 | Train MAE: 0.4430
Epoch 5 | Val RMSE: 0.5288 | Val MAE: 0.4345
Epoch 6 | Train RMSE: 0.5388 | Train MAE: 0.4418
Epoch 6 | Val RMSE: 0.5316 | Val MAE: 0.4312
Epoch 7 | Train RMSE: 0.5370 | Train MAE: 0.4403
Epoch 7 | Val RMSE: 0.5260 | Val MAE: 0.4307
Epoch 8 | Train RMSE: 0.5364 | Train MAE: 0.4395
Epoch 8 | Val RMSE: 0.5263 | Val MAE: 0.4291
Epoch 9 | Train RMSE: 0.5354 | Train MAE: 0.4387
Epoch 9 | Val RMSE: 0.5261 | Val MAE: 0.4326
Epoch 10 | Train RMSE: 0.5342 | Train MAE: 0.4375
Epoch 10 | Val RMSE: 0.5246 | Val MAE: 0.4280
Epoch 11 | Train RMSE: 0.5331 | Train MAE: 0.4367
Epoch 11 | Val RMSE: 0.5234 | Val MAE: 0.4290
Epoch 12 | Train RMSE: 0.5319 | Train MAE: 0.4357
Epoch 12 | Val RMSE: 0.5225 | Val MAE: 0.4287
```

We can see that the model almost stopped learning after the first 2 epochs, with only marginal improvements after epoch 2, what is even more strange, that we didn't overfit at all, leading me to believe that the model is underfitting, or what is also just as likely, is that we have hit a plateau, as in the Kaggle competition page, we can clearly see that the winner got 0.413 test score, and according to the competition page is RMSE, meaning we are not that far away, and considering what tactics Kaggle winner normally deploy, its only natural that they would get better results compared to our more or less naïve solutions.

So while there may be a room for improvement, and I will play with the model for a bit before reporting my best scores, the model may have already saturated everything it can learn and its simply impossible to improve any further as there is nothing left to learn, which is also very possible considering that in our task at hand, we have to rely on human labeling, and despite their instructions saying to rate objectively, this task is still very open to interpretation meaning that different graders will yield different labels further corrupting our data.

Underfit test: as im unsure whether the model is really underfitting, or if the task is simply non solvable for this type of models, I will use a more complex architecture, including self attention layers:

```python
class CharBiLSTMAttnEncoder(nn.Module):  1 usage
    def __init__(self, vocab_size, embed_dim=64, hidden_dim=128, dropout=0.3):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        self.lstm = nn.LSTM(
            embed_dim,
            hidden_dim,
            num_layers=2,
            bidirectional=True,
            batch_first=True,
            dropout=dropout
        )

        self.attn = SelfAttention(hidden_dim)
        self.fc = nn.Linear(hidden_dim * 2,  out_features: 256)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        x = self.emb(x)             # [B, L, E]
        x, _ = self.lstm(x)         # [B, L, 2H]
        x = self.attn(x)            # [B, 2H]
        x = self.dropout(self.fc(x))
        return x
```

```python
class SiameseRegression(nn.Module):  1 usage
    def __init__(self, vocab_size):
        super().__init__()
        self.encoder = CharBiLSTMAttnEncoder(vocab_size)
        self.fc = nn.Sequential(
            nn.Linear( in_features: 512,  out_features: 128),
            nn.ReLU(),
            nn.Linear( in_features: 128,  out_features: 1)
        )

    def forward(self, s, t):
        es = self.encoder(s)
        et = self.encoder(t)
        return self.fc(torch.cat( tensors: [es, et],  dim: 1)).squeeze(1)
```

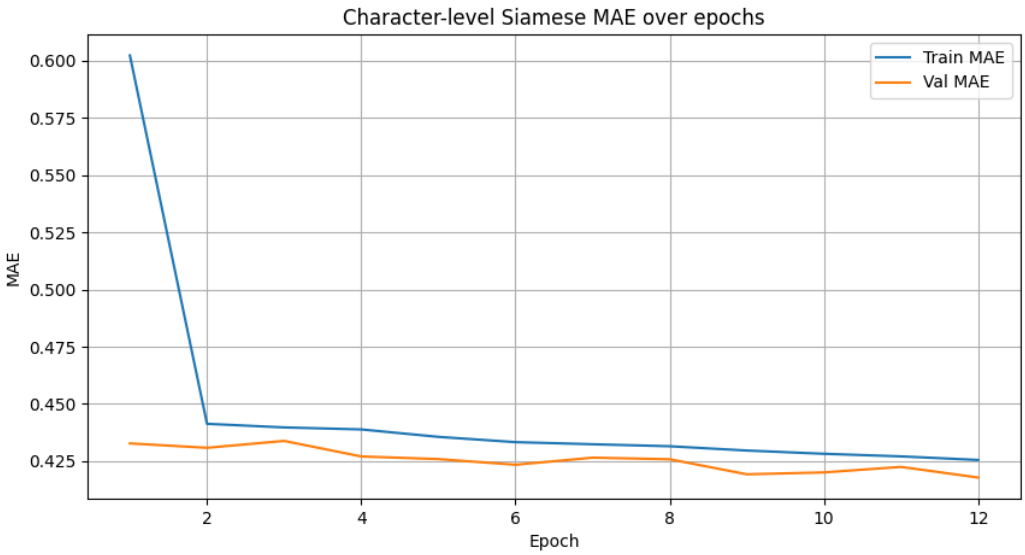
With the same hyper params we get:

Epoch 12 | Train RMSE: 0.5214 | Train MAE: 0.4255

Epoch 12 | Val RMSE: 0.5252 | Val MAE: 0.4179

FINAL TEST RMSE: 0.5357 | Test MAE: 0.4264

Total Neural Model Runtime (Train + Inference): 93.68 sec



meaning that there is no real room for improvments to be made, and since the cost is doubling the runtime, I will revert to fine tuning the simpler model and report on its final score:

BEST SCORE: one think I quickly realised as I was writing the report above, is that ideterminism and random noise would probably have a greater impact than hyper parameter tuning, simply because the optimization function already reaches the best it can almost immidietly, nonetheless I still have to try, as higher epoch have led me to a pattern of overfitting(marginal), I cut down the epochs as nothing other than noise is really learned there, and increased learning rate a bit just because.

Also another thing I now understand is that smaller batch size leads to a noisy gradient, which is great for generalization, but means less deterministic output, so I will try to increase it to see what happens

So far this is the best model I got:

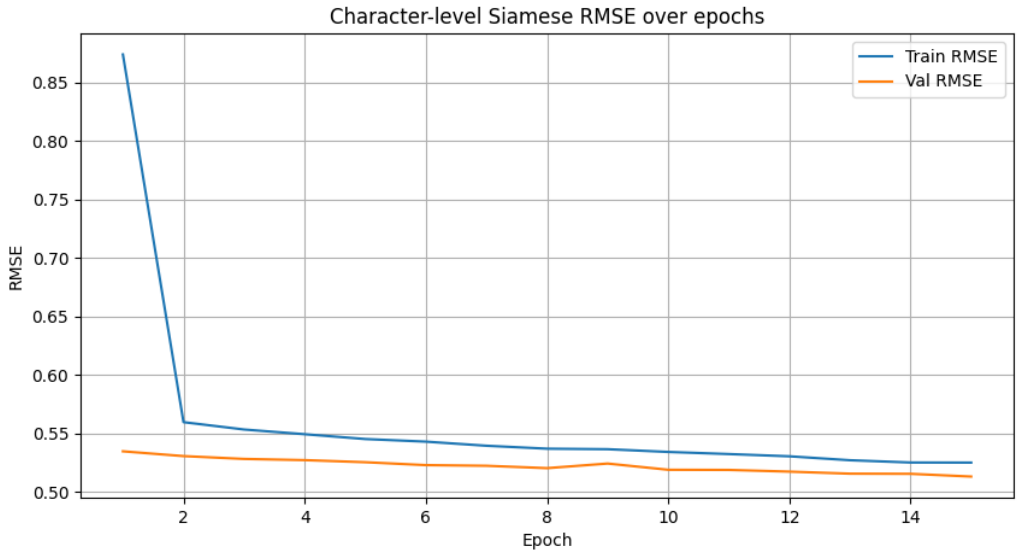
BATCH_SIZE = 512
EPOCHS = 12
LR = 1e-3
DROPOUT=0.4
MAX_SEARCH_LEN = 30
MAX_TITLE_LEN = 50

Epoch 12 | Train RMSE: 0.5308 | Train MAE: 0.4322

Epoch 12 | Val RMSE: 0.5165 | Val MAE: 0.4191

FINAL TEST RMSE: 0.5272 | Test MAE: 0.4285

Total Neural Model Runtime (Train + Inference): 36.21 sec



<u>BILTSM as a feature extractor:</u> TASK2_BILTSM_FE.py

FC Epoch 10 | Train RMSE: 2.0750 | Val RMSE: 2.0129

FC Head | Test RMSE: 2.0122 | Test MAE: 1.9406 | Runtime: 0.16s

Ridge | Test RMSE: 0.5278 | Test MAE: 0.4309 | Runtime: 0.07s

Random Forest Test RMSE: 0.5222 | MAE: 0.4270 | Runtime: 129.19s

Linear Regression Test RMSE: 0.5280 | MAE: 0.4309 | Runtime: 0.23s

MODEL 3: TASK1_SIAME_TRANSFORMER.py

It would be illegal to not try at least 1 transformer in this course, and considering just how fast this data trains, I think it's the perfect place to get familiar with transformer code, architecture, hardware effects and more.

Again just copy paste first attempt:

```
class CharTransformerEncoder(nn.Module):  1 usage
    def __init__(self, vocab_size, embed_dim=64, nhead=4, hidden_dim=128, num_layers=2, dropout=0.2):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        encoder_layer = nn.TransformerEncoderLayer(d_model=embed_dim, nhead=nhead, dim_feedforward=hidden_dim, dropout=dropout)
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        self.pool = nn.AdaptiveMaxPool1d(1)
        self.fc = nn.Linear(embed_dim,  out_features: 128)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        x = self.emb(x).permute(1,0,2)  # [seq, batch, embed_dim]
        x = self.transformer(x)         # [seq, batch, embed_dim]
        x = x.permute(1,2,0)            # [batch, embed_dim, seq]
        x = self.pool(x).squeeze(-1)
        x = self.dropout(F.relu(self.fc(x)))
        return x
```

```
class SiameseTransformerRegression(nn.Module):  1 usage
    def __init__(
        self,
        vocab_size,
        embed_dim=64,
        nhead=4,
        hidden_dim=128,
        num_layers=2,
        dropout=0.2
    ):
        super().__init__()
        # Our encoder is the CharTransformerEncoder
        self.encoder = CharTransformerEncoder(
            vocab_size=vocab_size,
            embed_dim=embed_dim,
            nhead=nhead,
            hidden_dim=hidden_dim,
            num_layers=num_layers,
            dropout=dropout
        )
        # Siamese head: concat outputs from search + ti
        self.fc = nn.Linear(128*2,  out_features: 1)  # 128

    def forward(self, s, t):
        """
        s: [batch, seq_len] search term indices
        t: [batch, seq_len] product title indices
        """
        es = self.encoder(s)  # [batch, 128]
        et = self.encoder(t)  # [batch, 128]
        combined = torch.cat( tensors: [es, et], dim=1)  #
        out = self.fc(combined).squeeze(1)     # [batch
        return out
```

```
model = SiameseTransformerRegression(
    vocab_size=len(char2idx),
    embed_dim=64,       # embedding siz
    nhead=4,            # attention hea
    hidden_dim=128,     # transformer f
    num_layers=2,       # number of tra
    dropout=DROPOUT     # same as your
).to(DEVICE)
```

with hyper params:

BATCH_SIZE = 512
EPOCHS =12
LR = 1e-3
DROPOUT=0.4
MAX_SEARCH_LEN = 30
MAX_TITLE_LEN = 50

```
Epoch 1 | Train RMSE: 0.7001 | Train MAE: 0.5523
Epoch 1 | Val RMSE: 0.7567 | Val MAE: 0.6569
Epoch 2 | Train RMSE: 0.5919 | Train MAE: 0.4825
Epoch 2 | Val RMSE: 0.7684 | Val MAE: 0.6697
Epoch 3 | Train RMSE: 0.5797 | Train MAE: 0.4725
Epoch 3 | Val RMSE: 0.7471 | Val MAE: 0.6501
Epoch 4 | Train RMSE: 0.5752 | Train MAE: 0.4689
Epoch 4 | Val RMSE: 0.7901 | Val MAE: 0.6907
Epoch 5 | Train RMSE: 0.5723 | Train MAE: 0.4674
Epoch 5 | Val RMSE: 0.7925 | Val MAE: 0.6932
Epoch 6 | Train RMSE: 0.5686 | Train MAE: 0.4636
Epoch 6 | Val RMSE: 0.7966 | Val MAE: 0.6968
Epoch 7 | Train RMSE: 0.5686 | Train MAE: 0.4632
Epoch 7 | Val RMSE: 0.8130 | Val MAE: 0.7120
```

Stopping early as its clearly not working as intendent, overfitting almost immidietly while also performing worse, I know that transformers are very data hungry, and ideally prefer longer sequences so that it can actually use the full attention.

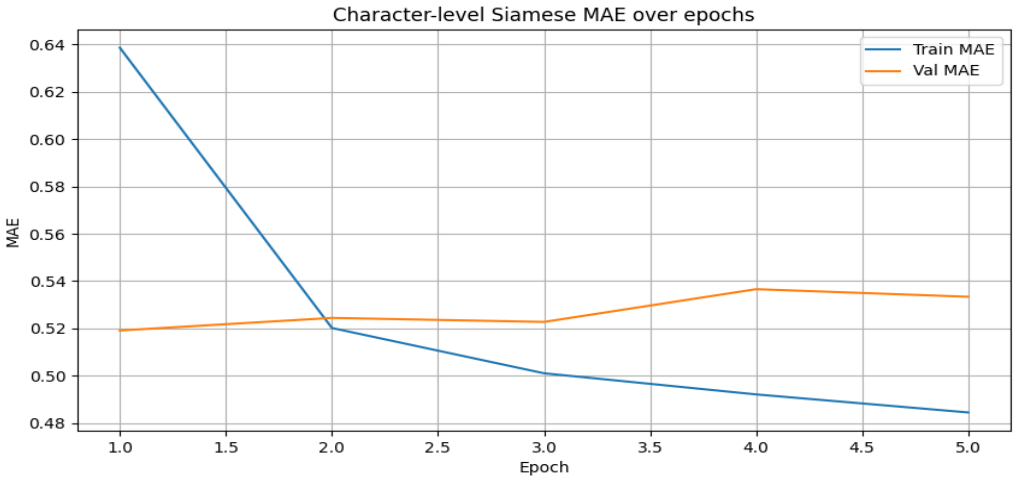
But at least its still super fast and not that memory intensive so I have the green light to continue.

Epoch 5 | Train RMSE: 0.5954 | Train MAE: 0.4845

Epoch 5 | Val RMSE: 0.6258 | Val MAE: 0.5334

FINAL TEST RMSE: 0.6304 | Test MAE: 0.5372

Total Neural Model Runtime (Train + Inference): 23.83 sec



we see and overfit+saturation, I may be able to make the model simpler to avoid over fitting, allong with smaller batch size, but I doubt it will noticebly improve or at least reach 0.53 RMSE test.

```
model = SiameseTransformerRegression(
    vocab_size=len(char2idx),
    embed_dim=32,       # embedding size
    nhead=2,            # attention head
    hidden_dim=64,     # transformer fee
    num_layers=2,       # number of tran
    dropout=DROPOUT     # same as your c
).to(DEVICE)
```

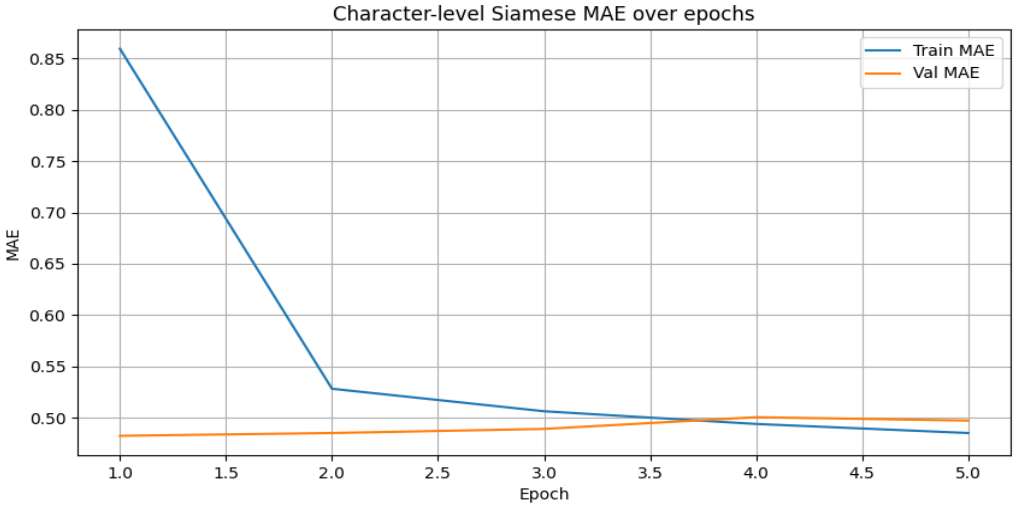
BATCH_SIZE = 128
EPOCHS = 5
LR = 5e-5
DROPOUT=0.4

Epoch 5 | Train RMSE: 0.5954 | Train MAE: 0.4850

Epoch 5 | Val RMSE: 0.5821 | Val MAE: 0.4971

FINAL TEST RMSE: 0.5791 | Test MAE: 0.4948

Total Neural Model Runtime (Train + Inference): 36.55 sec



we got a stable result with a pattern of starting to overfit, anything else I've tried ussualy was worse tha 0.59 test so I will stop at this.

<u>As a feature extractor:</u> TASK2_TRANS_FE.py

FC Epoch 10 | Train RMSE: 2.1019 | Val RMSE: 2.1008

FC Head | Test RMSE: 2.1014 | Test MAE: 2.0304 | Runtime: 0.44s

Ridge | Test RMSE: 0.5309 | MAE: 0.4344 | Runtime: 0.05s

Random Forest | Test RMSE: 0.5271 | MAE: 0.4317 | Runtime: 74.09s

Linear Regression | Test RMSE: 0.5308 | MAE: 0.4344 | Runtime: 0.21s

FINAL RESULTS Table: as to not explode the table, for feature extractor I will only include the best result

Model type	**Runtime** second	**Train** rmse	**Val** rmse	**Test** rmse	**Train** mae	**Val** mae	**Test** mae
Naïve baseline CountVectorizer+ Ridge regression	54.20 sec	0.2173	0.6367	0.7563	0.1575	0.4960	0.5930
Character embedding Cnn+siamese head	48.02 sec	0.4867	0.4937	0.5235	0.3950	0.3999	0.4243
Character embedding cnn + ridge	0.07	0.4634	0.4973	0.5327	0.3734	0.4018	0.4312
Character embedding BILTSM+siamese head	36.21 sec	0.5308	0.5165	0.5272	0.4322	0.4191	0.4285
Character embedding BILTSM+random forest	125.62s	0.4461	0.4927	0.5222	0.3683	0.4026	0.4270
Character embedding transformer+siamese head	23.83 sec	0.5954	0.6258	0.6304	0.4845	0.5334	0.5372
Character embedding Transformer+ random forest	80.73sec	0.4868	0.5068	0.5271	0.4005	0.4148	0.4317

Word embedding Cnn+siamese head	83.91 sec	0.4775	0.4860	0.5179	0.3880	0.3954	0.4233
BERT	631.48 sec	0.5183	0.5377	0.5406	0.4230	0.4166	0.4197
MINILM	153.31 sec	0.5288	0.5265	0.5307	0.4324	0.4201	0.4248

Conclusions: of course using gpt to better word some of them with correct terminology, but those are my conclusions just rephrased. Not all conclusions are rephrased that way, only those where I think I lack the terminology and expresive power.

1) Data-limited performance dominates this task: Just like in Task 1, there is a limit to what can be achieved with the current dataset. Factors such as subjective and inconsistent human labeling of relevance, length mismatch between search queries and product titles, a relatively small dataset (~74k samples), and class imbalance toward high relevance all contribute to this limitation. This is evident when comparing to the Kaggle leaderboard, where even top submissions achieve RMSE > 0.4 on a target range of [1, 3], corresponding to an average error of roughly 20%. In such scenarios, deep learning models may appear to underfit, when in reality they have saturated on the available data and cannot learn additional meaningful patterns.
2) **Plateauing models do not always indicate underfitting**:
When a model stops improving on both training and validation sets despite increased capacity, longer training, or lower learning rates,

this typically reflects a **data-limited ceiling** rather than true underfitting. The irreducible error in the dataset — arising from label noise and limited signal in the inputs — prevents the model from achieving perfect performance.

3) **Implications for modeling strategy**:
   In tasks with high irreducible error, further increasing model complexity is unlikely to yield meaningful gains. Instead, performance improvements are more likely to come from **data-centric approaches**: collecting more labeled data, improving label consistency, adding richer features (e.g., metadata or embeddings), or engineering higher-quality input representations.
4) **Limited overfitting in data-saturated tasks**: In tasks with a natural data ceiling, models may struggle to overfit. In my experiments, increasing model capacity did not yield meaningful improvements, nor did it lead to significant overfitting, which may seem counterintuitive. In our task, this is likely due to very short sequences, which make it difficult for the encoder to learn distinct representations for each input.
   At first, this seems surprising — the FC head should be able to overfit almost any dataset if it has enough capacity. However, the encoder's embeddings cannot fully "memorize" the labels because they remain too general. As a result, the head is limited by the quality of the embeddings and cannot fully overfit, meaning the model plateaus on both training and validation performance. Essentially, the bottleneck is the encoder's inability to produce fine-grained, label-specific representations.
5) **Transformers like big and long data, and are very sensitive to hyper parameters**. In my transformer, small changes to hyper params have led to high spikes in performance, from my observation, for smaller datasets like ours, it really prefers smaller batch sizes and lower learning rate.
6) **Transformers are fast to overfit.** While my BILTSM was unable to overfit not matter the capacity increase, my transformer was able to overfit in 2 epochs, and even after reducing its size by a factor of 2, it was still overfitting after 5 epochs.

7) **FC heads vs Siamese heads in transfer learning**, you cant just retrain an fc head on a given encoder, as an fc head is still very linear in its core, its most of the time unable to learn new embedings from scratch, meaning that it has to be trained alongside the encoder to be able to properly use its encodings, adapt and update the encoder to its liking.
8) **indeterminism / noise in training**: in some configurations, random noise and ideterminism have a more meaningfull effect than the hyper parameters, its especially obesrvable in tasks where we have data-limited performance, where the model hits a plateau and unable to learn anything new, and the main changing factors are things like seed and noise.
9) **Batch size effects**: complementing the previous conclusion, my expirements have shown that increasing the batch size often leads to a more stable and deterministic learning, and that's because larger batches produce less noisy gradient further boosting stability. But in return with too clean of a gradient, it's a lot easier to memorize as there is no random noise to obscure and throw the model off.
10) **Encoders as a bottleneck:** not really a new conclusion, as I've learned it already in the previous assigment, but its goodto remember that the model can only get as good the encoder itself, the encoder is the heart of the model, with it determining if the model overfits by giving too clear of an embedding, if it succeeds by giving strong and general embedding, or if it plateaus by giving not representative enough of an encoding.
11) Classical ML with embeddings Ridge, Linear Regression, and Random Forest performed roughly equally well, and hit a performance ceiling imposed by the encoder embeddings. The fact that these classical models plateaued at the same RMSE as neural FC heads reinforced the idea of encoder saturation: the model cannot extract more meaningful signal from the data.
12) Data-centric vs model-centric improvements In saturated tasks, further increasing model complexity doesn't help. Meaningful improvements would require better data, richer features, or more consistent labeling, not just bigger models.